

突破 Coding Agent 能力天花板

基于 Context Graph 的记忆架构演进

WU Zihan

January 30, 2026

1. 技术演进：从向量检索到 Context Graph

行业演进路线

第一代 (2020-2022): 向量 RAG

- ▶ 向量检索 + 文档切片
- ▶ 痛点：无结构、无时序、无关系

第二代 (2023): GraphRAG (Microsoft)

- ▶ 静态知识图谱 + 社区摘要
- ▶ 痛点：批量构建、无法增量更新

第三代 (2024-): Context Graph

- ▶ 时序感知 + Schema-free + 增量更新
- ▶ 专为 Agent Memory 设计

核心定位

Context Graph 通用知识图谱

而是为 LLM Agent 设计的记忆层架构

本质：存储“AI 应该记住什么”，而非“世界是什么样”

关键技术差异

维度	传统 KG	Context Graph
Schema	预定义	运行时自适应
更新	批量 ETL	毫秒级增量
时间	单时间戳	双时序追溯
输出	结构化数据	Token-aware

性能数据 (Zep Benchmark)

- ▶ DMR 准确率：94.8% (vs RAG 35.3%)
- ▶ LongMemEval 提升：+18.5%
- ▶ 检索延迟：300ms P95 (vs RAG 1s)
- ▶ Token 消耗：降低 85-93%

核心优势：Schema-free 降低本体工程成本，双时序支持决策追溯，增量更新适配研发迭代。

2. 核心方案：Context Graph 可生长记忆架构

方案本质：为 LLM Agent 设计的工作记忆层

借鉴 Zep/Graphiti (记忆存储) 与 LangGraph (流程控制)，构建 Coding 专用的 Schema-free 时序图谱。

架构设计：显式化“经验路径”

- 1. 节点 (Nodes) - 沉淀痕迹
 - ▶ Episode: 对话/操作流 (时序有效期)
 - ▶ Entity: 代码实体 (运行时推断类型)
 - ▶ Insight: 经验胶囊 (主动抽象规则, 如“X 错误 → Y 解法”)
- 2. 有向边 (Directed Edges) - 锁定逻辑
 - ▶ NEXT (时序): 记录操作流, 复用成功路径
 - ▶ CAUSES (因果): 根因分析, 多跳推理
 - ▶ DEPENDS_ON: 代码静态依赖 (AST)
- 3. 双时序建模: 事实有效期 + 系统记录时间, 支持“当时为什么这样决策”的追溯。

研发场景适配

场景	Context Graph 解法
需求变更	Schema-free 自适应
决策追溯	双时序可查历史
多人协作	跨 session 共享
代码 + 文档	Episode → Entity 统一

差异化竞争力

Code KG (静态) + Context Graph (动态)

- ▶ Code KG: “代码结构是什么” (AST、调用关系)
- ▶ Context Graph: “为什么这样写” (决策历史、需求演变)

同时理解代码 + 背后决策, 形成壁垒。

3. 竞争格局与战略价值

技术路线选择

传统动态 KG 路线 (不推荐)

- ▶ 需预定义本体, Schema 变更代价高
- ▶ 批量 ETL, 无法适配快速迭代
- ▶ 定位模糊: "又一个知识图谱应用"

Context Graph 路线 (推荐)

- ▶ 技术代际领先: 对齐 2024-2025 行业范式
- ▶ Schema-free 降低建模成本
- ▶ 产品故事清晰: "AI Agent 时代的记忆架构"
- ▶ 生态兼容: LangChain、LlamaIndex、MCP

竞品对标

- ▶ 传统工具厂商 (RAG): 我们有关系推理 + 时序追溯
- ▶ AI 创业公司 (通用 Memory): 我们深耕研发场景
- ▶ 大厂 (暴力长 context): 我们更省 token、可私有部署

落地路线

Phase 1 (MVP): 基于 Graphiti 开源版快速验证

- ▶ 场景: SWE-bench Lite
- ▶ 目标: 主动总结失败模式, 不再重复犯错

Phase 2 (产品化): 定制研发领域 Entity Types

- ▶ 场景: C/C++ 遗留系统
- ▶ 技术: AST 依赖 + Context Graph 协同

Phase 3 (差异化): 多 Agent 协作 + 知识迁移

- ▶ 跨项目迁移成熟图谱
- ▶ 项目级知识沉淀与复用

业务价值

降本: Token 节约 85-93% (Benchmark)
增效: Bug 解决率 +20-30%
资产化: 算力 → 企业私有知识资产

构建具备"经验记忆"的 **Autonomous Engineer**, 非工具而是进化的智能体。

4. 头部公司选择与行业洞察

头部公司 Memory 方案对比

公司	方案	路线
OpenAI	Dual-layer	算力暴力
Google	2M Context	算力暴力
Anthropic	文件系统式	透明可编辑
Zep (YC)	Context Graph	架构创新
Letta	虚拟内存 OS	架构创新

关键洞察

- ▶ 大厂路线：算力暴力（超长 context、超大模型）
- ▶ 创业路线：架构创新（Context Graph）
- ▶ 对企业级系统：架构创新更可控、可私有部署、成本可预测

开源生态支持

- ▶ Graphiti (Zep): 时序图谱参考实现
- ▶ LangGraph: StateGraph 流程控制
- ▶ Mem0: 向量 + 图混合方案
- ▶ GraphRAG: 社区检测 + 层级摘要

技术趋势三大转变

1. 被动检索 → 主动学习
 - ▶ RAG: 被动挖掘已有知识
 - ▶ Context Graph: 主动总结经验规律
2. 概率匹配 → 确定性推理
 - ▶ 向量检索: "猜你喜欢"
 - ▶ 图谱遍历: 沿依赖链精准定位
3. 单次查询 → 知识迁移
 - ▶ 传统: 每次重新检索
 - ▶ Context Graph: 组合已知模式解决未见问题

战略判断

Context Graph 是 AI Agent Memory 的下一代范式

- ▶ 时间窗口：2024-2025 行业转折点
- ▶ 机会：大厂尚未形成壁垒
- ▶ 壁垒：领域 Schema + 数据飞轮